Instituto Tecnológico Superior De Rio verde

Alumno: Luis Alejandro Domínguez Ramírez

Materia: Programación orientada a objetos

Profesor: ING. José de Jesús Collazo Reyes

Grupo: 2-A

Temas: Herencia en POO

Fecha: 28/04/2026

INTRODUCCION

La herencia en C++ es un concepto muy importante dentro de la programación orientada a objetos. Básicamente, sirve para crear nuevas clases a partir de otras que ya existen, aprovechando sus características y comportamientos. Es como si una clase “hija” heredara lo que tiene una clase “padre”, evitando tener que escribir todo desde cero.

En otras palabras, la herencia nos permite reutilizar código. Por ejemplo, si tenemos una clase llamada Vehículo con atributos como velocidad o marca, podemos crear otras clases como Auto o Bicicleta que hereden esas características. Así, no necesitamos volver a declarar esos atributos en cada clase nueva, sino que simplemente los heredamos.

En C++, la herencia se utiliza con dos puntos (:) después del nombre de la clase hija. Por ejemplo:

```
classAuto:publicVehiculo{};
```

Aquí estamos diciendo que la clase Auto hereda de Vehiculo. La palabra public indica el tipo de herencia, que define cómo se van a comportar los atributos y métodos heredados.

Algo importante es que la clase hija no solo hereda, sino que también puede agregar cosas nuevas o modificar lo que ya existe. Por ejemplo, puede tener funciones propias o incluso cambiar el comportamiento de una función heredada usando algo llamado *override*. Esto hace que el programa sea más flexible y adaptable.

También existen diferentes tipos de herencia en C++, como la herencia simple (una clase hereda de otra), la herencia múltiple (una clase hereda de varias) y la herencia jerárquica (varias clases heredan de una sola). Cada una se usa dependiendo de lo que necesite el programa.

En conclusión, la herencia es muy útil porque facilita la organización del código, evita repeticiones y permite crear programas más claros y fáciles de mantener. En la vida real, esto ayuda mucho cuando se desarrollan sistemas grandes, ya que se puede trabajar de manera más ordenada y eficiente.

EXPLICACON DEL SISTEMA

El problema consiste en diseñar un sistema para un videojuego en el que existen diferentes tipos de personajes: guerrero, mago y arquero. Todos los personajes comparten ciertas características en común como el nombre, el nivel y la vida, pero cada uno tiene habilidades distintas que los hacen únicos dentro del juego.

El objetivo es organizar estos personajes de forma lógica utilizando el concepto de herencia en programación orientada a objetos, de manera que se puedan reutilizar las características comunes y al mismo tiempo permitir que cada tipo tenga comportamientos propios. Además, se debe considerar un personaje especial que combina habilidades de otros personajes.

IDENTIFICACION DE CLASES

Para resolver el problema, se identifica una clase principal llamada **Personaje**, que contiene los atributos comunes de todos los personajes.

A partir de esta clase base, se derivan las siguientes clases:

- Guerrero
- Mago
- Arquero

Cada una de estas clases representa un tipo específico de personaje con habilidades diferentes. También se considera la creación de una clase adicional llamada PersonajeEspecial, que combina características de los demás personajes.

DISELO DE LA SOLUCION

Clases

Para resolver el problema, se definió una clase base llamada **Personaje**, de la cual se derivan las siguientes clases:

- **Guerrero**

- **Mago**
- **Arquero**
- **PersonajeEspecial**

La clase **Personaje** contiene las características generales, mientras que las demás representan tipos específicos con habilidades diferentes.

Atributos

Clase base (Personaje):

- nombre
- nivel
- vida

Clases derivadas:

- **Guerrero:**
  - fuerza
  - defensa
- **Mago:**
  - mana
  - inteligencia
- **Arquero:**
  - precisión
  - agilidad
- **PersonajeEspecial:**
  - habilidadesMixtas
  - experienciaExtra

Métodos

Clase base (Personaje):

- atacar()
- defender()
- mostrarInfo()

Clases derivadas:

- **Guerrero:**
  - atacar() (más fuerte)
  - defensaEspecial()
- **Mago:**
  - atacar() (con magia)
  - lanzarHechizo()
- **Arquero:**
  - atacar() (a distancia)
  - dispararFlecha()
- **PersonajeEspecial:**
  - atacar() (combinado)
  - habilidadEspecial()

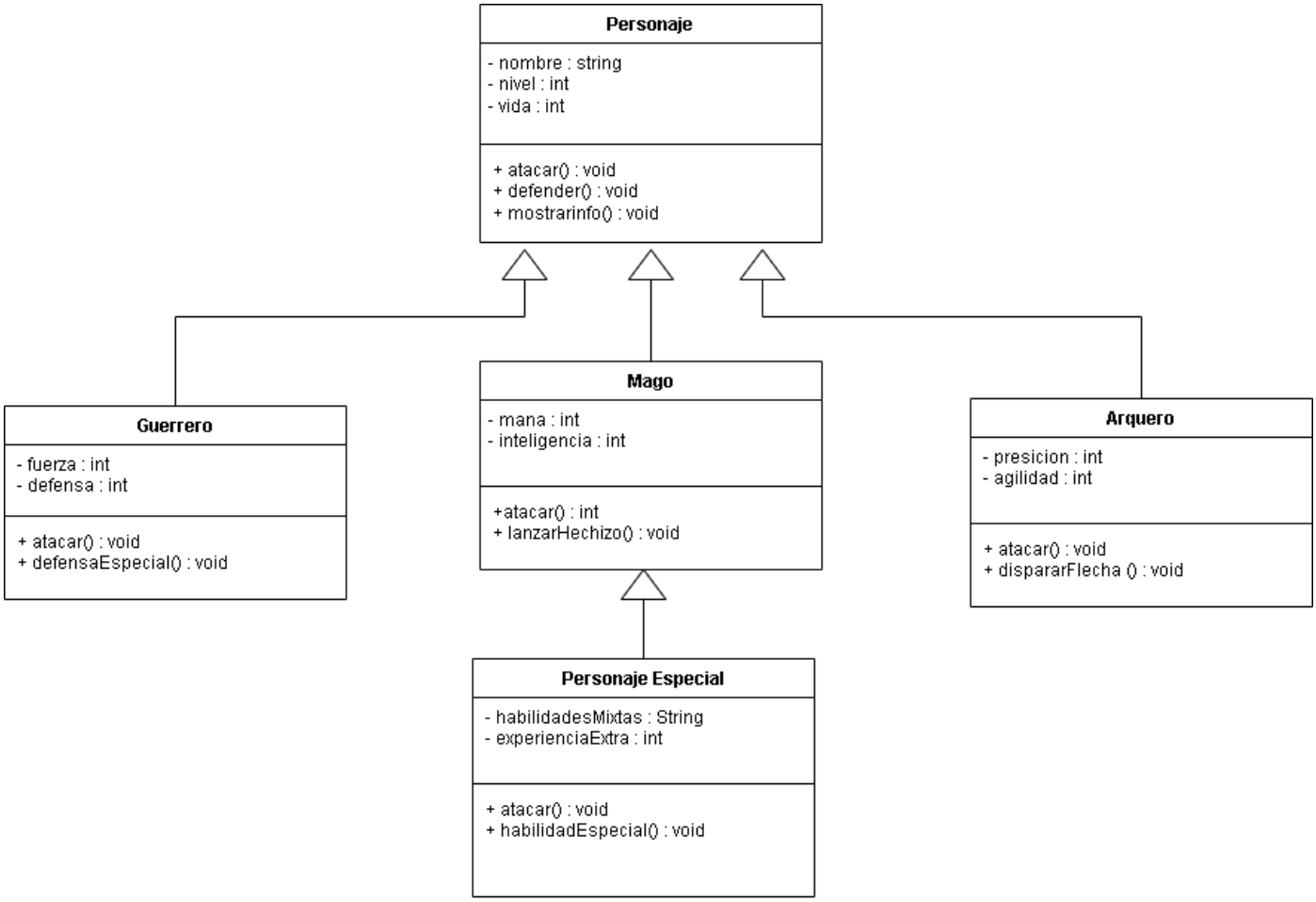
JUSTIFICACION DE DESICIONES

Se decidió utilizar una clase base (Personaje) para agrupar los atributos y métodos comunes, lo que permite reutilizar código y mantener el sistema organizado. Esto evita repetir información en cada clase y facilita futuras modificaciones.

Las clases derivadas se crearon para representar las diferencias entre los personajes, permitiendo que cada uno tenga habilidades específicas sin perder las características generales.

En el caso del **PersonajeEspecial**, se optó por no usar herencia múltiple para evitar complejidad en el diseño. En su lugar, se decidió que herede de la clase base y combine habilidades mediante atributos y métodos propios, logrando un equilibrio entre funcionalidad y simplicidad.

DIAGRAMA



EXPLICACION PERSONAL

Decidí organizar mi solución de esta manera porque me pareció más lógico usar una clase base llamada Personaje**,** ya que todos los personajes comparten características como el nombre, nivel y vida. A partir de esa clase, crear clases derivadas como Guerrero, Mago y Arquero permite reutilizar esas características y al mismo tiempo agregar habilidades específicas a cada uno. Esto hace que el diseño sea más ordenado y fácil de entender.

Lo más difícil fue decidir cómo representar al personaje especial, ya que tenía que combinar habilidades de otros personajes. Dudé si usar herencia múltiple o no, pero al final opté por una solución más sencilla para mantener el diseño claro y evitar complicaciones.

CONCLUSION

En esta actividad aprendí cómo funciona la herencia en la programación orientada a objetos y cómo se puede usar para organizar mejor un sistema. Entendí que es importante identificar qué elementos comparten características y cómo se pueden reutilizar para evitar repetir información, haciendo el diseño más claro y ordenado.

Este conocimiento sirve en la vida real para desarrollar programas más estructurados y fáciles de mantener, especialmente en proyectos grandes como videojuegos, aplicaciones o sistemas empresariales. También ayuda a pensar de forma lógica y a resolver problemas de manera más organizada antes de empezar a programar.